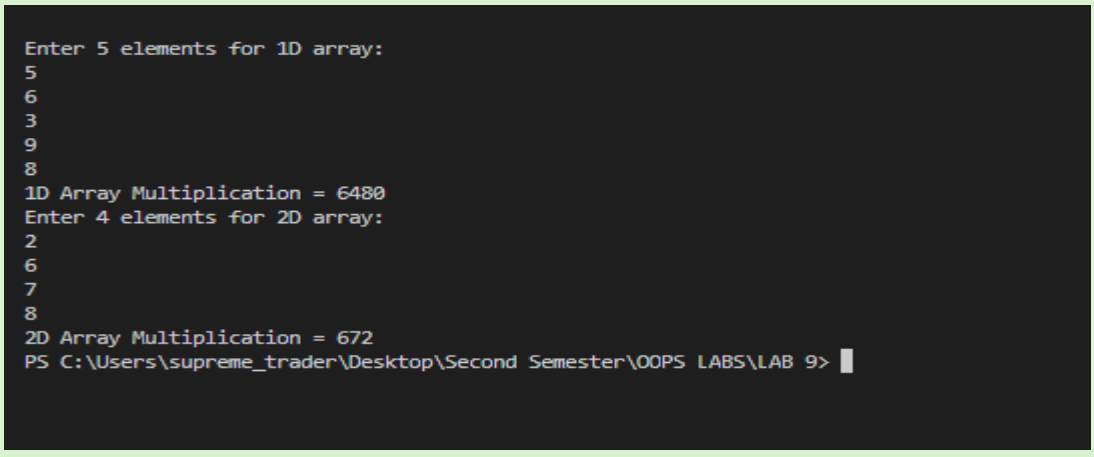
# QUESTION NO#01:

```
#include <iostream>
using namespace std;

class ArrayMultiplier {
public:
  virtual void calculate() = 0;
};

class ArrayMultiplier1D : public ArrayMultiplier {
private:
  int arr[5];

public:
  ArrayMultiplier1D() {
    for (int i = 0; i < 5; i++) {
      cin >> arr[i];
    }
  }

  void calculate() {
    int product = 1;

    for (int i = 0; i < 5; i++) {
      product *= arr[i];
```

```
    }

    cout << "1D Array Multiplication = " << product << endl;
  }
};

class ArrayMultiplier2D : public ArrayMultiplier {
private:
  int arr[2][2];

public:
  ArrayMultiplier2D() {
    for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
        cin >> arr[i][j];
      }
    }
  }

  void calculate() {
    int product = 1;

    for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
        product *= arr[i][j];
      }
```

```
    }

    cout << "2D Array Multiplication = " << product << endl;
  }
};

int main() {
  cout << "Enter 5 elements for 1D array:" << endl;
  ArrayMultiplier1D obj1;
  obj1.calculate();

  cout << "Enter 4 elements for 2D array:" << endl;
  ArrayMultiplier2D obj2;
  obj2.calculate();

  return 0;
}
```

## OUTPUT:



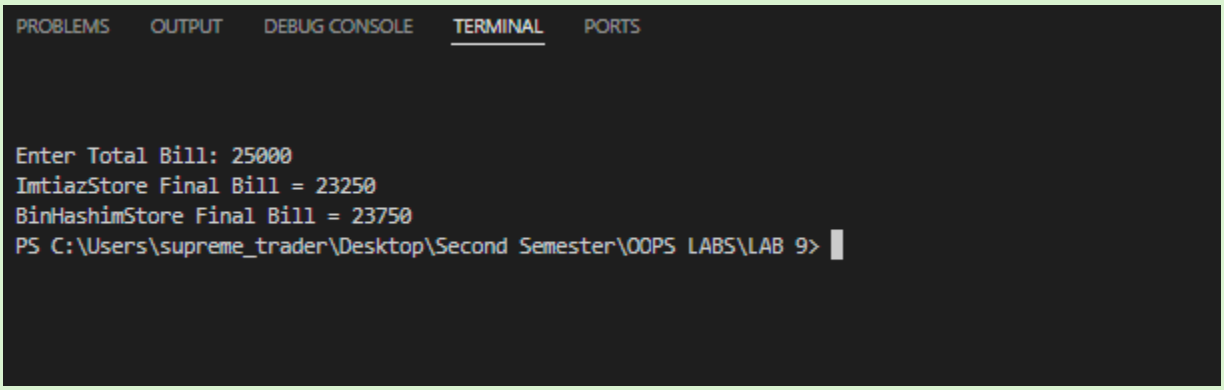
# QUESTION NO#02:

```
#include <iostream>
using namespace std;

class Store {
protected:
   float total_bill;

public:
   Store(float bill) {
      total_bill = bill;
   }

   virtual void calculateBill() = 0;
};

class ImtiazStore : public Store {
public:
   ImtiazStore(float bill) : Store(bill) {}

   void calculateBill() {
      float discount = total_bill * 0.07;
      float final_bill = total_bill - discount;
```

```
        cout << "ImtiazStore Final Bill = " << final_bill << endl;
    }
};

class BinHashimStore : public Store {
public:
    BinHashimStore(float bill) : Store(bill) {}

    void calculateBill() {
        float discount = total_bill * 0.05;
        float final_bill = total_bill - discount;

        cout << "BinHashimStore Final Bill = " << final_bill << endl;
    }
};

int main() {
    float bill;

    cout << "Enter Total Bill: ";
    cin >> bill;

    ImtiazStore obj1(bill);
    BinHashimStore obj2(bill);

    obj1.calculateBill();
```

```
    obj2.calculateBill();


    return 0;
}
```

## OUTPUT:




## QUESTION NO#03:

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
protected:
    int carId;
    string brand;
    string model;
```

```
public:
    Vehicle(int id, string b, string m) {
        carId = id;
        brand = b;
        model = m;
    }

    virtual bool isAvailable() = 0;
    virtual void rent() = 0;
    virtual void returnVehicle() = 0;
    virtual void display() = 0;

    virtual ~Vehicle() {
    }
};

class Car : public Vehicle {
private:
    bool available;

public:
    Car(int id, string b, string m) : Vehicle(id, b, m) {
        available = true;
    }

    bool isAvailable() {
```

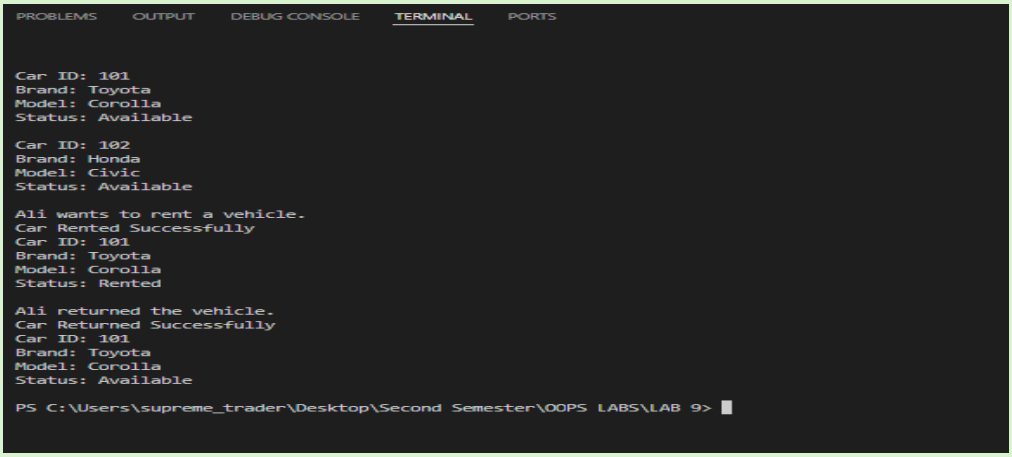
```
    return available;
}

void rent() {
    if (available) {
        available = false;
        cout << "Car Rented Successfully" << endl;
    }
    else {
        cout << "Car Not Available" << endl;
    }
}

void returnVehicle() {
    available = true;
    cout << "Car Returned Successfully" << endl;
}

void display() {
    cout << "Car ID: " << carId << endl;
    cout << "Brand: " << brand << endl;
    cout << "Model: " << model << endl;

    if (available)
        cout << "Status: Available" << endl;
    else
```

```
        cout << "Status: Rented" << endl;

    cout << endl;
  }
};

class RentalSystem {
public:
  void rentVehicle(Vehicle *v) {
    if (v->isAvailable()) {
      v->rent();
    }
    else {
      cout << "Vehicle Already Rented" << endl;
    }
  }

  void returnVehicle(Vehicle *v) {
    v->returnVehicle();
  }
};

class Customer {
private:
  string name;
```

```
public:
    Customer(string n) {
        name = n;
    }

    void rentVehicle(RentalSystem &rs, Vehicle *v) {
        cout << name << " wants to rent a vehicle." << endl;
        rs.rentVehicle(v);
    }

    void returnVehicle(RentalSystem &rs, Vehicle *v) {
        cout << name << " returned the vehicle." << endl;
        rs.returnVehicle(v);
    }
};

int main() {
    Vehicle *vehicles[2];

    vehicles[0] = new Car(101, "Toyota", "Corolla");
    vehicles[1] = new Car(102, "Honda", "Civic");

    RentalSystem system;

    Customer c1("Ali");
```

```
    for (int i = 0; i < 2; i++) {
        vehicles[i]->display();
    }

    c1.rentVehicle(system, vehicles[0]);

    vehicles[0]->display();

    c1.returnVehicle(system, vehicles[0]);

    vehicles[0]->display();

    for (int i = 0; i < 2; i++) {
        delete vehicles[i];
    }

    return 0;
}
```

# OUTPUT:



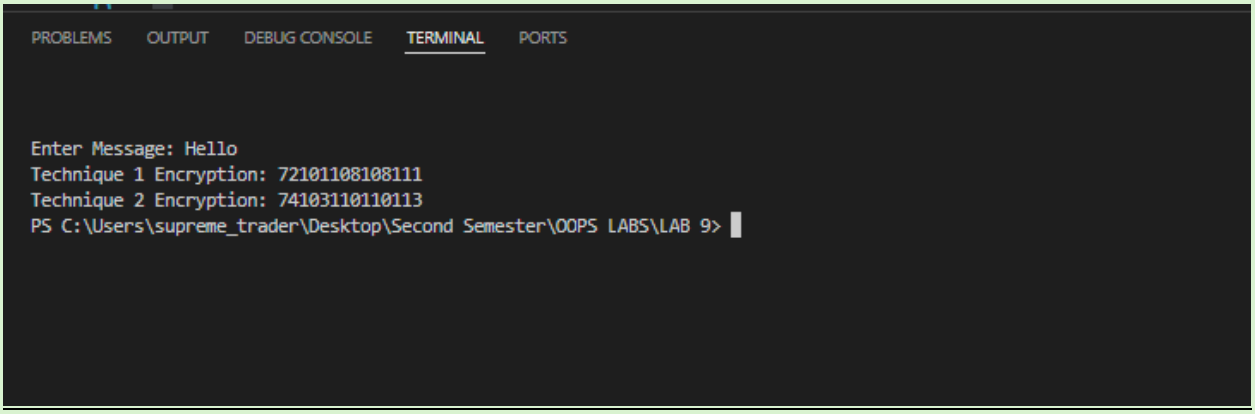
# QUESTION NO#04:

```
#include <iostream>
#include <string>
using namespace std;

class EncryptionTechnique {
public:
   virtual void encrypt(string message) = 0;
};

class EncryptionTechnique1 : public EncryptionTechnique {
public:
   void encrypt(string message) {
      string result = "";

      for (int i = 0; i < message.length(); i++) {
         result += to_string((int)message[i]);
      }

      cout << "Technique 1 Encryption: " << result << endl;
   }
};

class EncryptionTechnique2 : public EncryptionTechnique {
```

```
public:
    void encrypt(string message) {
        string result = "";

        for (int i = 0; i < message.length(); i++) {
            result += to_string((int)message[i] + 2);
        }

        cout << "Technique 2 Encryption: " << result << endl;
    }
};

int main() {
    string message;

    cout << "Enter Message: ";
    getline(cin, message);

    EncryptionTechnique *e1;
    EncryptionTechnique *e2;

    e1 = new EncryptionTechnique1();
    e2 = new EncryptionTechnique2();

    e1->encrypt(message);
    e2->encrypt(message);
```

```
    delete e1;
    delete e2;

    return 0;
}
```

## OUTPUT:




## QUESTION NO#05:

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

class DecryptionTechnique {
public:
    virtual void decrypt(string code) = 0;
};
```

```
class DecryptionTechnique1 : public DecryptionTechnique {
public:
   void decrypt(string code) {
      stringstream ss(code);
      int value;
      string result = "";

      while (ss >> value) {
         result += char(value);
      }

      cout << "Technique 1 Decryption: " << result << endl;
   }
};

class DecryptionTechnique2 : public DecryptionTechnique {
public:
   void decrypt(string code) {
      stringstream ss(code);
      int value;
      string result = "";

      while (ss >> value) {
         result += char(value - 2);
      }
```

```
        cout << "Technique 2 Decryption: " << result << endl;
    }
};

int main() {
    string code1, code2;

    cout << "Enter Encrypted String for Technique 1: ";
    getline(cin, code1);

    cout << "Enter Encrypted String for Technique 2: ";
    getline(cin, code2);

    DecryptionTechnique *d1;
    DecryptionTechnique *d2;

    d1 = new DecryptionTechnique1();
    d2 = new DecryptionTechnique2();

    d1->decrypt(code1);
    d2->decrypt(code2);

    delete d1;
    delete d2;
```

```
    return 0;
}
```

## OUTPUT: